

LectureDoc2



Autor: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de, Raum 149B

Folien: <https://delors.github.io/LectureDoc2/example-slides.en.rst.html>
Fehler melden: <https://github.com/Delors/delors.github.io/issues>

1. Introduction

This slide set primarily serves as a testbed for various features of the *LectureDoc2* system. It also demonstrates how to use the system to create slides for a lecture.

Supplemental Information and Presenter Notes[1]

You first have to enter the master password (press M and then enter Demo!) to see the presenter notes.

This is additional information that is not shown in the main text.

Some more text.

This is some more additional information that is not shown in the main text.

[illegible]

[1] Supplemental information and presenter notes are not immediately shown on the slide.

Decks and Cards

A deck is a collection of cards.

Where each card "replaces" the previous cards during the presentation belonging to the same deck.

This card contains a simple note. Where the height of the deck as a whole is determined by the tallest card.

 Note

This is a simple note.

The Tallest One

Above the crowd, I stand so high, A bridge between the ground and sky. I see the world in a broader frame, Yet hear the jokes—they're all the same.

—Jan. 2025 ChatGPT (Prompt: I need a short poem about being the tallest one.)



Decks can be nested and can overlay each other!

However, a card with a nested deck is not allowed to also use floating elements (e.g. notes). In general, the use of floating elements in combination with overlays is discouraged.

The first sentence of the first card in the nested deck.

A sentence in between.

The last sentence of the first card in the nested deck.

※ Hint

This is the last meaningful card in the nested deck. The next two ones are a technical detail.

 Note

This is another simple note.

XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX --
XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX---
XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX----
XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX --
XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX---
XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX----
XXXXXXXX XXXXXXXX

※ Hint

This is the last card in the top-level deck.

Stories

Stories are used for content that should appear in a stepwise manner and which may scroll content out of the view.

1. This is the first step.
2. This is the second step.
3. This is the third step.
4. This is the fourth step.
5. This is the fifth step.
6. This is the sixth step.
7. This is the seventh step.
8. This is the eighth step.
9. This is the ninth step.
10. This is the tenth step.
11. This is the eleventh step.
12. This is the twelfth step.
13. This is the thirteenth step.
14. This is the fourteenth step.
15. This is the fifteenth step.

Some monospaced text.

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}  
  
print("Hello, World!")  
  
fn main() {  
    println!("Hello, World!");  
}  
  
const std = @import("std");  
  
pub fn main() void {  
    std.debug.print("Hello, World!\n", .{});  
}
```

Scrollables

A scrollable is a container whose content does not fit into the available space of a slide. During the presentation the content can be scrolled by the presenter and scrolling is relayed in secondary windows.

```
1  /* A small library to encrypt and decrypt strings using AES-GCM and PBKDF2.
2   *
3   * Based on code found at: https://github.com/themikefuller/Web-Cryptography
4   *
5   * License: BSD-3-Clause
6   */
7  export {
8    decrypt as decryptAESGCMGBKDF,
9    encrypt as encryptASEGCMGBKDF
10 }
11
12 async function encrypt(plaintext, password, iterations) {
13
14   const encodedPlaintext = new TextEncoder().encode(plaintext);
15   const encodedPassword = new TextEncoder().encode(password);
16
17   const pass = await crypto.subtle.importKey(
18     'raw',
19     encodedPassword,
20     { "name": "PBKDF2" },
21     false,
22     ['deriveBits']);
23
24   const salt = crypto.getRandomValues(new Uint8Array(32));
25   const iv = crypto.getRandomValues(new Uint8Array(12));
26
27   const keyBits = await crypto.subtle.deriveBits(
28     {
29       "name": "PBKDF2",
30       "salt": salt,
31       "iterations": iterations,
32       "hash": { "name": "SHA-256" }
33     },
34     pass,
35     256);
36
37   const key = await crypto.subtle.importKey(
38     'raw',
39     keyBits, { "name": "AES-GCM" },
40     false,
41     ['encrypt']);
42
43   const enc = await crypto.subtle.encrypt(
44     {
45       "name": "AES-GCM",
46       "iv": iv
47     },
```

```

48     key,
49     encodedPlaintext);
50
51     const iterationsB64 = btoa(ounds.toString());
52
53     const saltB64 = btoa(Array.from(new Uint8Array(salt)).map(val => {
54         return String.fromCharCode(val)
55     })).join(''));
56
57     const ivB64 = btoa(Array.from(new Uint8Array(iv)).map(val => {
58         return String.fromCharCode(val)
59     })).join(''));
60
61     const encB64 = btoa(Array.from(new Uint8Array(enc)).map(val => {
62         return String.fromCharCode(val)
63     })).join(''));
64
65     return iterationsB64 + ':' + saltB64 + ':' + ivB64 + ':' + encB64;
66 };
67
68 async function decrypt(encrypted, password) {
69
70     const parts = encrypted.split(':');
71     const rounds = parseInt(atob(parts[0]));
72
73     const salt = new Uint8Array(atob(parts[1]).split('').map(val => {
74         return val.charCodeAt(0);
75     }));
76
77     const iv = new Uint8Array(atob(parts[2]).split('').map(val => {
78         return val.charCodeAt(0);
79     }));
80
81     const enc = new Uint8Array(atob(parts[3]).split('').map(val => {
82         return val.charCodeAt(0);
83     }));
84
85     const encodedPassword = new TextEncoder().encode(password);
86     const pass = await crypto.subtle.importKey(
87         'raw',
88         encodedPassword,
89         { "name": "PBKDF2" },
90         false,
91         ['deriveBits']);
92
93     const keyBits = await crypto.subtle.deriveBits(
94         {
95             "name": "PBKDF2",
96             "salt": salt,
97             "iterations": rounds,
98             "hash": {
99                 "name": "SHA-256"
100             }
101         },

```



```
102     pass,  
103     256);  
104  
105     let key = await crypto.subtle.importKey(  
106         'raw',  
107         keyBits, { "name": "AES-GCM" },  
108         false,  
109         ['decrypt']);  
110  
111     let dec = await crypto.subtle.decrypt(  
112         {  
113             "name": "AES-GCM",  
114             "iv": iv  
115         },  
116         key,  
117         enc);  
118  
119     return (new TextDecoder()).decode(dec);  
120 };
```

Scrollables with explicit height!^[2]

A scrollable can have an explicit height that will be used for the slide view.

```
1  /**
2   * Adds an event listener to the scrollable element that fires when the element
3   * is scrolled. In that case, the event is sent to the specified channel to
4   * make secondary windows aware of the scrolling event in the primary window.
5   *
6   * The data is sent using the {@link postMessage} method where the msg is the event title
7   * and the data is a two element array where the first element is the id of the
8   * element that is being scrolled and the second element is the current scrollTop.
9   *
10  * The primary window is always the window that user interacts with. The secondary
11  * is every other window showing the same site.
12  *
13  * @param {Channel} channel - The channel that will be used to send the event.
14  * @param {string} eventTitle - The title of the event that will be sent to the channel.
15  *                               title has to be unique w.r.t. to the channel.
16  * @param {HTMLElement} scrollableElement - The element that is being scrolled.
17  * @param {string} id - The id of the element that is being scrolled.
18  */
19  export function addScrollingEventListener(channel, eventTitle, scrollableElement, id) {
20      // We will relay a scroll event to a secondary window, when there was no
21      // more scrolling for at least TIMEOUTms. Additionally, if there is already an
22      // event handler scheduled, we will not schedule another one.
23      //
24      // If we would directly relay the event, it may be possible that it will
25      // result in all kinds of strange behaviors, because we cannot easily
26      // distinguish between a programmatic and a user initiated scroll event.
27      // (Using window blur and focus events didn't work reliably.)
28      // This could result in a nasty ping-pong effect where scrolling between
29      // two different position would happen indefinitely.
30      const TIMEOUT = 50;
31      let lastEvent = undefined;
32      let eventHandlerScheduled = false;
33      scrollableElement.addEventListener("scroll", (event) => {
34          lastEvent = new Date().getTime();
35          function scheduleEventHandler(timeout) {
36              setTimeout(() => {
37                  const currentTime = new Date().getTime();
38                  if (currentTime - lastEvent < TIMEOUT) {
39                      scheduleEventHandler(TIMEOUT - (currentTime - lastEvent));
40                      return;
41                  }
42                  postMessage(channel, eventTitle, [id, event.target.scrollTop]);
43                  // console.log(eventTitle + " " + id + " " + event.target.scrollTop);
44                  eventHandlerScheduled = false;
45              }, timeout);
46          };
47          if(!eventHandlerScheduled) {
48              eventHandlerScheduled = true;
49              scheduleEventHandler(TIMEOUT);
```

```
50     }  
51     }, {passive: true});  
52 }
```

This is a scrollable that extends to the bottom of the slide -100px to leave some space for the footer.

```
1  export function getTopAndBottomMargin(e) {  
2      const style = window.getComputedStyle(e);  
3      return parseInt(style.marginTop) + parseInt(style.marginBottom);  
4  }  
5  export function getLeftAndRightMargin(e) {  
6      const style = window.getComputedStyle(e);  
7      return parseInt(style.marginLeft) + parseInt(style.marginRight);  
8  }  
9  export function getLeftAndRightPadding(e) {  
10     const style = window.getComputedStyle(e);  
11     return parseInt(style.paddingLeft) + parseInt(style.paddingRight);  
12 }  
13 export function getLeftAndRightMarginAndPadding(e) {  
14     return getLeftAndRightMargin(e) + getLeftAndRightPadding(e);  
15 }  
16  
17 export function postMessage(channel, msg, data) {  
18     channel.postMessage([msg, data]);  
19 }
```

[2] Just a footer.

Simple column-based Layouts

One way to create a very simple multi-column layout consists of a list with the class `columns`.

Example

Default layout:

The first column.
1. a nested list.

The second column.
1. a nested list.

Left-aligned layout:

The first column. The second column.
1. a nested list. 1. a nested list.

Evenly-spaced layout:

The first column.
1. a nested list.

The second column.
1. a nested list.

Advanced Slide Layouts

Using Grids it is possible to design advanced slide layouts.

When you don't specify a specific layout for a grid a simple multi-column layout is used.

```
1 .. grid::
2 :class: very-light-gray-background
3
4 .. cell::
5
6     Using Grids it is possible to
7     design advanced slide layouts.
8
9 .. cell::
10
11 .. code:: rst
12 :class: copy-to-clipboard
13
14     **The Code**
```

Math

Adding math (e.g. $a^2 + b^2 = c^2$) to a slide is done using the math directive or role.

$$e = mc^2$$

Alternatively, it is often possible and easier to just use Unicode symbols:

Poor Man's Math: $e = mc^2$.

Example

Adding math (e.g. `:math: a^2+b^2=c^2`) to a slide is done using the math directive or role.

`.. math::`

$$e = mc^2$$

Poor Man's Math: $e = mc^2$.

Tables

.highlight-identical-cells-on-hover

	r	s	t	u	v	w	x	y	z
a	1	2	3	4	5	6	7	8	9
b	4	5	6	7	8	9	1	2	3
c	7	8	9	1	2	3	4	5	6

.highlight-row-on-hover

	r	s	t	u	v	w	x	y	z
a	1	2	3	4	5	6	7	8	9
b	4	5	6	7	8	9	1	2	3